

AnomaMind: Agentic Time Series Anomaly Detection with Tool-Augmented Reasoning

Xiaoyu Tao, Yuchong Wu, Mingyue Cheng, Ze Guo, and Tian Gao

State Key Laboratory of Cognitive Intelligence, University of Science and Technology of China
Hefei, Anhui Province, China

txytiny@mail.ustc.edu.cn, wu3091455397@mail.ustc.edu.cn, mycheng@ustc.edu.cn,
gz1504921411@mail.ustc.edu.cn, ustc25gt@mail.ustc.edu.cn

Abstract—Time series anomaly detection is critical in many real-world applications, where effective solutions must localize anomalous regions and support reliable decision-making under complex settings. However, most existing methods frame anomaly detection as a purely discriminative prediction task with fixed feature representations, rather than an evidence-driven diagnostic process. As a result, they often struggle when anomalies exhibit strong context dependence, diverse patterns, or domain shifts across datasets. To address these challenges, we propose AnomaMind, an agentic time series anomaly detection framework that reformulates anomaly detection as a sequential decision-making process. AnomaMind operates through a coarse-to-fine workflow that first localizes suspicious intervals, then constructs diagnostic evidence through tool interaction, and finally refines anomaly decisions through self-reflection. The workflow is supported by a toolkit box that combines knowledge memory and numerical diagnostics: visual anomaly patterns mined from training data and domain knowledge provide contextual guidance, while statistical, value-based, change-based, and region-level operators provide measurable evidence for verification. AnomaMind further adopts a hybrid inference mechanism in which general-purpose models handle flexible reasoning, tool invocation, and refinement, while a detection-specific policy is optimized with rule-based rewards for parsable outputs, F1-score alignment, and false-positive control. Extensive experiments under both in-domain and cross-domain settings demonstrate that AnomaMind consistently improves anomaly detection performance and enhances generalization across heterogeneous anomaly patterns, validating the effectiveness of tool-augmented reasoning for anomaly detection.¹

Index Terms—Anomaly Detection, Agentic Workflow, Tool-augmented Reasoning

I. INTRODUCTION

Time series anomaly detection (TSAD) plays a critical role in many real-world systems, where detected anomalies often trigger downstream actions such as alarms, diagnosis, or intervention [1]. In such settings, effective anomaly detection requires more than assigning anomaly scores to individual observations. Practical solutions need to localize anomalous intervals, interpret diagnostic evidence, and support decision-making under evolving contextual conditions with limited or uncertain information. As time series data become increasingly diverse and context-dependent, these requirements call for evidence-driven and decision-oriented detection paradigms beyond pure predictive accuracy [2].

Despite substantial progress [3], most existing TSAD methods remain model-centric. They typically formulate anomaly detection as a discriminative prediction task [4], [5] based on fixed feature representations and predefined scoring mechanisms. This design limits their flexibility during inference, especially when anomalous patterns vary across temporal resolutions, evolve over time, or are strongly influenced by contextual conditions [6]. More fundamentally, these methods lack mechanisms to adaptively construct informative evidence, reason over intermediate results, and revise earlier decisions when new diagnostic information becomes available, which restricts their effectiveness in complex real-world settings, such as scenarios involving concept shift [7].

We argue that these limitations stem from a fundamental mismatch between how anomaly detection is commonly formulated and how it is carried out in practice [8]. In real-world scenarios, TSAD rarely reduces to a single scoring step, rather than a fixed one-shot classification problem. Instead, it typically unfolds as a sequential decision-making process, where analysts progressively narrow down suspicious intervals, examine informative evidence, and update judgments based on intermediate findings [9]. Decisions made at earlier stages influence what evidence is gathered next and how subsequent analyses are conducted across temporal scales and contextual assumptions [10]. However, conventional model-centric formulations are not designed to represent such diagnostic procedures [11]. These observations motivate moving beyond isolated predictive models toward agentic time series systems that organize anomaly detection as a system-level reasoning and decision process, integrating evidence acquisition, analysis, and refinement within an adaptive workflow.

While this perspective motivates a shift toward agentic time series systems, realizing such an anomaly detection process in practice remains challenging. First, effective anomaly reasoning requires reusable analytical resources that provide both contextual guidance and measurable verification signals [12]. An agent should recognize recurring anomaly patterns, retrieve domain knowledge, and compute numerical diagnostics without relying on hard-coded, task-specific pipelines [13]. Second, the detection workflow must be explicitly structured to support progressive decision-making, where coarse localization and fine-grained analysis are coherently integrated rather than treated as isolated stages [14]. Such a coarse-to-

¹The code is available at <https://github.com/Xiaoyu-Tao/AnomaMind-TS>.

fine workflow is essential for balancing efficiency, accuracy, and interpretability. Third, agentic inference requires a stable optimization mechanism. Although general-purpose models are flexible for reasoning and tool coordination, they often lack the inductive biases and task-specific learning signals needed for reliable anomaly decisions [15]. Addressing this issue calls for a hybrid inference paradigm that combines flexible reasoning with task-specific decision learning under explicit and verifiable objectives.

To address these challenges, we propose AnomaMind, an agentic time series anomaly detection framework that explicitly formulates anomaly detection as a sequential decision-making process. AnomaMind operates through a structured workflow that incrementally constructs diagnostic evidence and refines anomaly decisions over multiple steps. Rather than relying on fixed feature inputs, AnomaMind progressively localizes anomalous intervals in a coarse-to-fine manner and augments detection through a toolkit box that integrates knowledge memory and numerical diagnostics. The knowledge memory includes visual anomaly patterns mined from training data, domain knowledge, and tool descriptions, while the numerical-evidence toolkit provides statistical, value-based, change-based, and region-level operators for measurable verification. Crucially, AnomaMind adopts a hybrid inference mechanism: general-purpose models handle tool invocation and self-reflective refinement, while a detection-specific policy is optimized through rule-based rewards that encourage valid parsing, F1-score alignment, and false-positive control. This design enables AnomaMind to integrate flexible reasoning, contextual grounding, and task-specific optimization in complex anomaly detection scenarios.

Our main contributions are summarized as follows:

- We reformulate TSAD as a sequential decision-making problem, moving beyond static discriminative detection toward an evidence-driven diagnostic process.
- We propose **AnomaMind**, an agentic framework built around a coarse-to-fine workflow and a toolkit box that combines knowledge memory and numerical diagnostics.
- We design a hybrid inference mechanism in which general-purpose models perform flexible reasoning and tool orchestration, while a detection-specific policy is optimized with verifiable rule-based rewards.
- We conduct extensive experiments in both in-domain and cross-domain TSAD settings, showing that AnomaMind improves detection performance under dataset-specific training and maintains strong generalization across heterogeneous anomaly patterns through agentic, tool-augmented reasoning.

II. RELATED WORK

In this section, we first review conventional anomaly detection methods and then introduce recent advances in LLM-based TSAD.

A. Conventional Anomaly Detection

Existing TSAD methods span diverse modeling paradigms [3]. Early statistical approaches detect anomalies by assuming that normal time series follow distributional or structural patterns [16]. Classical detectors model components such as trend and seasonality and identify anomalies by thresholding residuals between observations and model-based predictions, as exemplified by ARIMA-based residual analysis [17]. Density-based approaches estimate data distributions or relative sample densities and treat sparse observations as anomalies [18]. These methods are interpretable but rely on handcrafted assumptions. To alleviate this limitation, deep learning methods adopt representation learning to model temporal patterns flexibly [19]. Reconstruction-based approaches, such as USAD [20], learn representations of normal time series using autoencoders and regard large reconstruction errors as anomalies. Forecasting-based approaches predict future values from historical observations and detect anomalies through large deviations between predicted and actual values. Recent recurrent architectures such as xLSTMAD [21] further improve anomaly detection by combining encoder-decoder temporal modeling with reconstruction objectives.

B. LLM-based Anomaly Detection

More recently, LLM-based approaches have been explored for TSAD [22]. Early studies mainly convert numerical sequences into textual representations and feed them into pre-trained LLMs for zero-shot or few-shot anomaly identification. Representative methods such as prompt-based GPT-4 detectors [23] show that LLMs can capture coarse abnormal patterns without task-specific training by leveraging rich knowledge. Related forecasting studies indicate that LLMs can benefit from explicit temporal patterns and semantic descriptions [24], while vision-language models can interpret complex time-series behaviors through wavelet and recurrence representations [25]. However, these detection-only or representation-driven methods largely remain static input-output mappings, with limited temporal-structure modeling and diagnostic reasoning. In contrast, training-based strategies introduce task-specific adaptation, typically through parameter-efficient fine-tuning or learnable adapters, to better align LLMs with anomaly detection objectives [6]. For example, AnomalyLLM [26] distills anomaly-aware representations from a pre-trained LLM into a student network and detects anomalies via teacher-student feature discrepancies. More recently, tool-augmented LLM frameworks further reframe TSAD as a multi-step decision-making process. For instance, ARGOS [27] employs LLM-based agents to iteratively generate, validate, and refine anomaly detection rules with external tools and contextual features.

III. PRELIMINARIES

We first formulate time series anomaly detection from a sequential decision-making perspective and then summarize the reasoning process that motivates agentic detection.

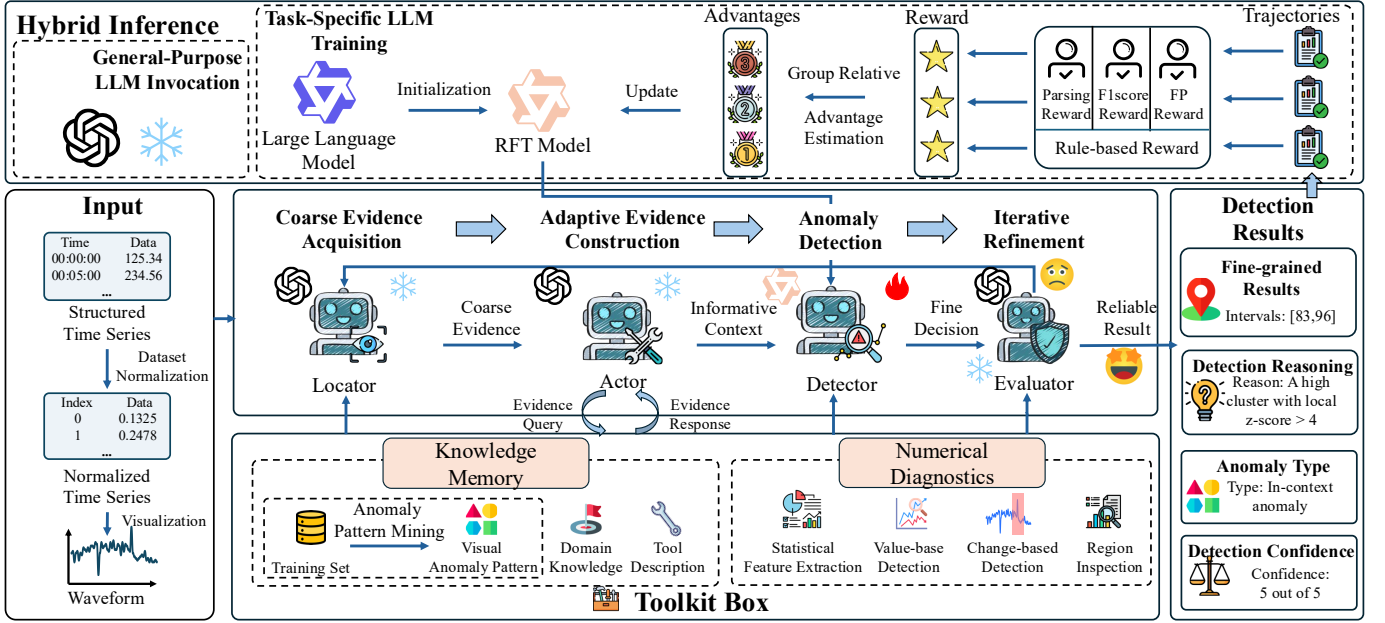


Fig. 1. Overview of AnomaMind, an agentic time series anomaly detection framework that adopts a structured coarse-to-fine workflow built upon a toolkit box and supported by hybrid inference.

A. Problem Formulation

We formalize TSAD as a *sequential decision-making process*. Given an input time series $\mathbf{x}_{1:T} = \{x_1, x_2, \dots, x_T\}$ and optional contextual information $\mathbf{c}_{1:T}$, detection proceeds over steps $k = 1, \dots, K$ rather than relying on a fixed one-shot feature input. At step k , the detection process maintains a state s_k that contains the original observations, contextual signals, intermediate analysis results, and evidence produced by external tools. Based on this evolving state, an action a_k is selected from a structured action space, such as refining candidate regions, invoking diagnostic analyses, or updating anomaly judgments. The action returns an observation or evidence item o_k , which is incorporated into the next state s_{k+1} . After a finite decision horizon or a stopping condition, the process outputs a set of detected anomalous intervals $\hat{S}$. This formulation enables anomaly decisions to be progressively refined through multi-step evidence acquisition and revision.

B. Thinking Process for Anomaly Detection

In practice, anomaly detection is rarely a single-pass scoring operation. It often begins with coarse screening to identify potentially suspicious intervals while filtering evidently normal regions [28]. Targeted diagnostic analysis is then applied to these intervals by examining local statistical irregularities, temporal patterns, and contextual inconsistencies [7], [29]. Since different intervals may require different evidence, early judgments are provisional and may be progressively revised when additional information becomes available, rather than relying on fixed thresholds or one-size-fits-all scoring rules across operational contexts [9], [30]. This process naturally suggests an agentic formulation in which anomaly detection

is organized as candidate localization, evidence construction, and reliable decision refinement.

IV. THE PROPOSED ANOMAMIND

Building on the sequential decision-making formulation introduced earlier, we present AnomaMind, an agentic framework for time series anomaly detection. AnomaMind operationalizes anomaly detection through adaptive feature preparation, tool-augmented reasoning, and iterative decision refinement, as detailed in the following sections.

A. Framework Overview

Figure 1 illustrates the overall framework of AnomaMind. Built upon a toolkit box with knowledge memory and numerical diagnostics, AnomaMind provides structured support for perception, evidence construction, and reasoning. It organizes anomaly detection as a progressive decision workflow, including coarse-grained interval localization, fine-grained detection, and iterative refinement, where different roles coordinate for sequential decision-making. To support this workflow, AnomaMind adopts a hybrid inference mechanism that combines general-purpose LLM-based reasoning with task-specific model learning. The LLM performs autonomous tool invocation and self-reflection, while anomaly detection decisions are optimized through reinforcement learning with workflow-level feedback. The following subsections detail each component.

B. Toolkit Box Construction

As shown at the bottom of Figure 1, the toolkit box is denoted as $\mathcal{B} = \{\mathcal{K}, \mathcal{N}\}$, where $\mathcal{K}$ represents knowledge memory for semantic and procedural guidance, and $\mathcal{N}$ represents numerical diagnostics for measurable anomaly verification. Rather than embedding all functionality into a single

TABLE I
ROLE-SPECIFIC USE OF KNOWLEDGE MEMORY AND NUMERICAL DIAGNOSTICS IN ANOMAMIND.

Role	Workflow Stage	Knowledge Memory	Numerical Diagnostics	Output
Locator	Coarse Evidence Acquisition	Visual Anomaly Pattern; Domain Knowledge	Not directly used	Coarse Evidence
Actor	Adaptive Evidence Construction	Domain Knowledge; Tool Description	Invoke and construct	Informative Context
Detector	Anomaly Detection	Domain Knowledge	Use accumulated diagnostics	Fine Decision
Evaluator	Iterative Refinement	Domain Knowledge	Verify diagnostic consistency	Reliable Result

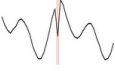
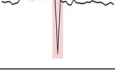
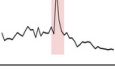
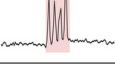
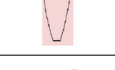
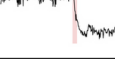
Anomaly Type	Description	Example
Local Dip	A brief, sharp dip at one or a few points that breaks the local pattern, but stays within the normal overall range.	
Local Spike	A brief, sharp spike at one or a few points that breaks the local pattern, but stays within the normal overall range.	
Global Dip	A sharp downward dip that falls far below the rest of the segment, is an extreme outlier on the global scale, and then returns to baseline.	
Global Spike	A sharp upward spike that rises far above the rest of the segment, is an extreme outlier on the global scale, and then returns to baseline.	
Spike Burst	Several sharp upward spikes occur close together in a short interval, while the series before and after stays near a quiet baseline.	
Transient Plateau Up	The series jumps to a higher level, stays on a plateau for a while, then returns to the original baseline.	
Transient Plateau Down	The series drops to a lower level, stays on a plateau for a while, then returns to the original baseline.	
Level Shift Down	A sustained downward shift in the series to a new, lower mean level, without reverting to the previous baseline.	

Fig. 2. Representative time-series anomaly types.

model, AnomaMind allows different roles to query the toolkit according to their stage-specific needs. The system maintains candidate intervals $\mathcal{S}^{(k)} = \{[l_j, r_j]\}_{j=1}^{M_k}$, accumulated numerical diagnostics $\mathcal{Z}_{1:k}$, and retrieved contextual knowledge $\mathcal{C}_{1:k}$ throughout the workflow. The four roles summarized in Table I are represented as locator $\mathcal{L}$, actor $\mathcal{A}$, detector $\mathcal{D}_\theta$, and evaluator $\mathcal{E}$. Formally, a toolkit query at step k is written as:

$$o_k = \mathcal{B}_{a_k}(\mathbf{x}_{1:T}, \mathcal{S}^{(k)}, \mathcal{C}_{1:k}), \quad (1)$$

where $\mathcal{B}_{a_k} \in \mathcal{B}$ is the selected toolkit component and o_k is either retrieved knowledge or newly computed numerical diagnostics. If o_k is knowledge, it is appended to $\mathcal{C}_{1:k}$; if it is numerical diagnostics, it is appended to $\mathcal{Z}_{1:k}$.

a) *Knowledge Memory*: The knowledge memory $\mathcal{K}$ corresponds to the left part of the toolkit box in Figure 1. It stores three types of reusable knowledge:

$$\mathcal{K} = \{\mathcal{K}_{vap}, \mathcal{K}_{dom}, \mathcal{K}_{tool}\}, \quad (2)$$

where $\mathcal{K}_{vap}$ contains visual anomaly patterns mined from training data, $\mathcal{K}_{dom}$ contains domain knowledge, and $\mathcal{K}_{tool}$ contains tool descriptions. As illustrated in Figure 2, the domain knowledge provides anomaly-type names, textual descriptions, and representative examples, which help explain the semantics of common time-series anomaly patterns. Given the current observation, memory produces contextual knowledge $\mathcal{C}^{(k)} = \mathcal{K}(\mathbf{x}_{1:T}, o^{(k)})$ to support the decisions of the locator, actor, detector, and evaluator.

The visual anomaly patterns in $\mathcal{K}_{vap}$ are constructed offline through anomaly pattern mining over the training set, corresponding to the ‘‘Anomaly Pattern Mining’’ block in Figure 1. As summarized in Algorithm 1, we extract annotated anomalous segments, group them by length, map each segment to an 18-dimensional descriptor, and select representative visual prototypes through length-stratified clustering. These prototypes are used as visual contextual inputs during detection. For each segment u_i , the descriptor $\phi(u_i) \in \mathbb{R}^{18}$ covers length and ratio, anomaly statistics, contextual deviation, boundary contrast, and variation features. Within each length group g , we apply KMeans to cluster descriptors:

$$\{C_{g,1}, \dots, C_{g,H}\} = \text{KMeans}(\{\phi(u_i) \mid u_i \in \mathcal{U}^g\}). \quad (3)$$

For each cluster $C_{g,h}$, the segment closest to the centroid is selected as the visual prototype. These prototypes form the visual anomaly pattern set $\mathcal{K}_{vap}$, which complements the domain knowledge $\mathcal{K}_{dom}$ during context retrieval.

b) *Numerical Diagnostics*: The numerical-evidence toolkit $\mathcal{N}$ corresponds to the right part of the toolkit box in Figure 1. It contains four evidence operators:

$$\mathcal{N} = \{\mathcal{N}_{stat}, \mathcal{N}_{val}, \mathcal{N}_{chg}, \mathcal{N}_{reg}\}. \quad (4)$$

Here $\mathcal{N}_{stat}$ computes descriptive statistics for a candidate segment, including local and global mean, variance, percentiles, maximum, minimum, and median values. $\mathcal{N}_{val}$ identifies magnitude outliers by normalizing segment values with global statistics and returning points that exceed an adaptive threshold. $\mathcal{N}_{chg}$ detects abrupt transitions by normalizing adjacent-point differences with global difference statistics and returning points with excessive changes. $\mathcal{N}_{reg}$ inspects the values within a specified index range, helping verify whether a candidate interval contains meaningful temporal evidence. For an interval $s_j = [l_j, r_j]$, a selected evidence operator outputs $z_j^{(k)} = \mathcal{N}_{a_k}(\mathbf{x}_{l_j:r_j})$, which is accumulated into $\mathcal{Z}_{1:k}$. These measurable signals provide the numerical basis.

Algorithm 1 Anomaly Pattern Mining

Require: Labeled training series $\mathcal{D}_{tr}$, number of clusters per length group H

Ensure: Visual anomaly patterns $\mathcal{K}_{vap}$

```
1: Extract anomalous segments  $\mathcal{U} = \{u_i\}_{i=1}^Q$  from  $\mathcal{D}_{tr}$ 
2: Partition  $\mathcal{U}$  into  $\{\mathcal{U}^{short}, \mathcal{U}^{medium}, \mathcal{U}^{long}\}$  by segment length
3: for each segment  $u_i \in \mathcal{U}$  do
4:   Compute descriptor  $\phi(u_i) \in \mathbb{R}^{18}$ 
5: end for
6: Initialize  $\mathcal{K}_{vap} \leftarrow \emptyset$ 
7: for each length group  $g \in \{short, medium, long\}$  do
8:   Cluster  $\{\phi(u_i) \mid u_i \in \mathcal{U}^g\}$  into  $\{C_{g,h}\}_{h=1}^H$  by KMeans
9:   for each cluster  $C_{g,h}$  do
10:    Select  $u_{g,h}^* = \arg \min_{u_i \in C_{g,h}} \|\phi(u_i) - \mu_{g,h}\|_2$ 
11:    Add  $u_{g,h}^*$  to  $\mathcal{K}_{vap}$ 
12:   end for
13: end for
14:
15: return  $\mathcal{K}_{vap}$ 
```

C. Coarse-to-Fine Detection Workflow

AnomaMind instantiates the general process as a structured coarse-to-fine workflow. The workflow is organized into four implementation stages, each corresponding to one role in Table I: coarse evidence acquisition by the locator, adaptive evidence construction by the actor, anomaly detection by the detector, and iterative refinement by the evaluator. These stages provide a unified mechanism for efficient localization, context-aware analysis, and accurate decision-making under complex and evolving conditions. One AnomaMind update is summarized as:

$$s_{k+1} = \mathcal{E}(s_k, a_k, o_k, \hat{\mathcal{Y}}^{(k)}), \quad (5)$$

where o_k is the knowledge or evidence returned by the toolkit box, and $\hat{\mathcal{Y}}^{(k)} = \{(\hat{s}_j, \hat{y}_j)\}$ denotes the interval-level decisions produced by $\mathcal{D}_\theta$. The final output is $\hat{\mathcal{S}} = \{\hat{s}_j \mid \hat{y}_j = 1\}$.

a) Coarse Evidence Acquisition: The workflow begins with coarse evidence acquisition, whose goal is to efficiently narrow down potentially anomalous regions from the full time series. At this stage, AnomaMind leverages visual perception to identify suspicious temporal intervals based on salient pattern irregularities, abrupt changes, or distributional shifts. Rather than performing precise anomaly judgments, this stage prioritizes recall and uncertainty awareness, producing a set of coarse candidate intervals that are likely to contain abnormal behavior. By filtering out evidently normal regions early on, coarse evidence acquisition significantly reduces the subsequent search space and enables focused analysis. This stage is represented as $\mathcal{S}^{(0)} = \mathcal{L}(\mathbf{x}_{1:T}, \mathcal{K}_{vap}, \mathcal{K}_{dom})$, where visual anomaly patterns and domain knowledge guide the locator toward high-recall coarse evidence.

b) Adaptive Evidence Construction: Given the candidate intervals identified in the coarse stage, AnomaMind proceeds

to adaptive evidence construction. In this stage, large language models autonomously orchestrate tool interactions to gather informative numerical and contextual evidence relevant to the detected candidates. The selection and invocation of tools are driven by intermediate observations and accumulated context, allowing different intervals to be analyzed using different diagnostic strategies. This adaptive process enables flexible feature preparation and evidence accumulation, ensuring that subsequent detection decisions are grounded in task-relevant and context-aware information rather than fixed feature pipelines. The actor chooses $a_k = \mathcal{A}(s_k, \mathcal{K}_{tool})$ according to the tool descriptions. If a_k selects a numerical operator, the evidence pool is augmented as $\mathcal{Z}_{1:k} = \mathcal{Z}_{1:k-1} \cup \{o_k\}$; if it selects knowledge memory, the contextual pool is augmented as $\mathcal{C}_{1:k} = \mathcal{C}_{1:k-1} \cup \{o_k\}$.

c) Anomaly Detection: Based on the constructed evidence, AnomaMind performs reasoning-based anomaly detection to produce fine-grained decisions. This stage is responsible for assessing whether candidate intervals exhibit anomalous behavior, given the numerical features, structural patterns, and contextual cues collected earlier. Unlike purely discriminative approaches, the detection policy is optimized through reinforcement learning under workflow-level feedback, allowing it to adapt to the sequential and interdependent nature of detection decisions. As a result, anomaly detection decisions reflect both local evidence and the broader diagnostic context accumulated across interaction steps. For each candidate interval $s_j \in \mathcal{S}^{(k)}$, the detector predicts:

$$\hat{y}_j = \mathcal{D}_\theta(s_j, \mathcal{Z}_{1:k}, \mathcal{C}_{1:k}), \quad \hat{y}_j \in \{0, 1\}. \quad (6)$$

d) Iterative Refinement: AnomaMind further incorporates an iterative refinement mechanism to evaluate and revise detection outcomes when necessary. Through self-reflection, intermediate decisions are examined for potential inconsistency, uncertainty, or insufficient evidence. If refinement is triggered, the workflow may return to earlier stages to acquire additional evidence, adjust analysis strategies, or reconsider candidate intervals. This iterative process enables principled decision revision and prevents premature or brittle conclusions, allowing anomaly detection to progressively converge toward more reliable results. The evaluator returns feedback $f_k = \mathcal{E}(s_k, \hat{\mathcal{Y}}^{(k)})$, which either terminates the workflow or updates the next state s_{k+1} for further evidence acquisition.

D. Hybrid Decision Mechanism

AnomaMind adopts a hybrid decision mechanism for stable inference across the coarse-to-fine detection workflow. The key idea is to separate general-purpose reasoning from task-specific anomaly decision learning, allowing each component to leverage its strengths. Specifically, AnomaMind combines (i) general-purpose LLMs for flexible reasoning and tool invocation and (ii) a detection-specific decision module optimized through reinforcement learning with workflow-level feedback.

a) General-Specialized Hybrid Reasoning: In AnomaMind, general-purpose LLMs are responsible for driving reasoning-intensive and structurally flexible components of

the workflow. These components include coarse-grained interval localization based on visual perception, autonomous invocation of feature extraction and diagnostic tools, and self-reflective assessment of intermediate detection results. Such stages require adaptive reasoning, dynamic control flow, and the ability to integrate heterogeneous evidence, which are well supported by general-purpose models without task-specific retraining. By contrast, these components do not require precise anomaly decision boundaries or strong task-specific inductive biases. Instead of enforcing rigid prediction objectives at this level, AnomaMind leverages the expressive reasoning capability of LLMs to flexibly explore suspicious regions, construct informative evidence, and guide subsequent decision steps. This design avoids over-constraining early-stage reasoning and enables the workflow to adapt to diverse anomaly patterns and contextual variations.

b) Reinforcement Learning for Anomaly Decision Learning: Before reinforcement learning, we conduct supervised fine-tuning to initialize the detection-specific policy with anomaly reasoning traces. This stage teaches the detector to interpret diagnostic context and generate interval-level outputs, while the RL stage further optimizes decision quality using parsing, F1-score, and false-positive rewards. While general-purpose reasoning is effective for evidence construction and workflow control, anomaly detection decisions demand task-specific learning signals and stable optimization for precise boundary localization. To this end, AnomaMind introduces a dedicated anomaly detection module whose decision policies are trained through reinforcement learning. Unlike prior approaches that apply multi-turn reinforcement learning to train the entire detection workflow end-to-end, AnomaMind restricts reinforcement learning to this local decision module [30], significantly reducing training complexity. Concretely, the trainable policy π_θ is attached to $\mathcal{D}_\theta$: its input is the accumulated diagnostic context $(\mathcal{S}^{(k)}, \mathcal{Z}_{1:k}, \mathcal{C}_{1:k})$, and its output is the interval decision set $\hat{\mathcal{Y}}^{(k)}$. The policy is optimized using rule-based rewards computed after evaluator feedback.

For each sampled detector output, the reward consists of three components:

$$r_k = \lambda_p r_k^{parse} + \lambda_f r_k^{F1} - \lambda_{fp} r_k^{FP}, \quad (7)$$

where λ_p , λ_f , and λ_{fp} control the relative weights. The parsing reward r_k^{parse} enforces a valid and machine-readable output format, requiring the detector to return anomaly intervals and binary labels according to the predefined schema. The F1-score reward r_k^{F1} measures the agreement between the predicted anomalous intervals $\hat{\mathcal{Y}}^{(k)}$ and the ground-truth labels, encouraging accurate localization and classification. The false-positive penalty r_k^{FP} penalizes normal regions that are incorrectly reported as anomalous, which is important for preventing overly aggressive anomaly predictions. These components jointly reward correct, parsable, and conservative anomaly decisions.

Given the diagnostic state $s_k = (\mathcal{S}^{(k)}, \mathcal{Z}_{1:k}, \mathcal{C}_{1:k})$, the final training objective is to maximize the expected cumulative

TABLE II
STATISTICS OF TIME SERIES ANOMALY DETECTION DATASETS WITH DIFFERENT ANNOTATION LEVELS.

Dataset	All Points	Train Points	Test Points	Anomaly Rate	Category
YAHOO	404,514	107,200	75,400	0.63%	Point & Segment
KPI	3,477,731	483,000	588,000	2.04%	Segment
IOPS	1,237,469	147,000	162,000	1.50%	Segment
WSD	1,943,551	89,000	114,000	0.60%	Segment

reward of the detection policy:

$$\max_{\theta} \mathbb{E}_{\hat{\mathcal{Y}}^{(k)} \sim \pi_\theta(\cdot | s_k)} \left[\sum_{k=1}^K r_k \right]. \quad (8)$$

By grounding the rewards in parsing validity, F1-score improvement, and false-positive control, the learned detection policy captures task-relevant inductive biases while remaining compatible with the surrounding reasoning-driven workflow.

V. EXPERIMENTS

In this section, we introduce the experimental settings and implementation details, followed by the main results, exploratory analyses, and finally a case study.

A. Experimental Settings

1) Datasets: Table II summarizes the statistics and annotation granularity of the datasets used in our experiments. We evaluate our method on four widely used TSAD benchmarks. Yahoo [31] contains real and synthetic time series derived from production traffic, with both point and segment anomalies. KPI and WSD [32] contain real-world key performance indicators collected from large-scale web services, while IOPS [33] includes system-level performance metrics reflecting service scale, quality, and machine health. KPI, IOPS, and WSD mainly contain segment-level anomalies, making them suitable for evaluating range-level anomaly localization.

2) Baselines: We compare our method with a diverse set of time-series anomaly detection approaches, covering statistical, classical machine learning, deep learning, foundation-model, and LLM-based paradigms. Following recent TSAD benchmark studies [1], [2], we include strong classical baselines that have shown competitive performance across univariate and multivariate settings. Statistical and classical baselines include DWT-MLEAD [34], FFT-AD², MatrixProfile [35], SR [36], and Sub-PCA [37], covering frequency-domain, wavelet-based, subsequence-distance, spectral residual, and PCA-based methods. Deep learning baselines include CNN [4], M2N2 [38], LSTMAD [39], TranAD [40], and OmniAnomaly [19], covering convolutional, recurrent, stochastic reconstruction, and adaptive detection settings. We further include foundation models, including TimesFM [41] and Chronos [42], which detect anomalies through forecasting deviations or direct anomaly inference. Finally, we compare with recent LLM-based approaches, including LLM-TSAD [15], LLM-AD [43], OFA [6], and ARGOS [27], which

²<https://github.com/TimeEval/TimeEval-algorithms>

TABLE III
COMPARISON OF ANOMALY DETECTION PERFORMANCE ON MULTIPLE BENCHMARK DATASETS. **BOLD** AND UNDERLINED VALUES INDICATE THE BEST AND SECOND-BEST PERFORMANCE, RESPECTIVELY.

Model	YAHOO				KPI				WSD				IOPS			
	F1	Best-F1	AUC-PR	Range-F1	F1	Best-F1	AUC-PR	Range-F1	F1	Best-F1	AUC-PR	Range-F1	F1	Best-F1	AUC-PR	Range-F1
DWT_MLEAD	0.0330	0.0912	0.0548	0.2495	0.1780	0.2064	0.1276	0.3689	0.0921	0.3135	0.2614	0.4108	0.1716	0.1722	0.1571	0.2489
FFT	0.1314	0.1967	0.1580	0.2911	0.2666	0.3046	0.1942	0.3727	0.1594	0.2303	0.1862	0.2921	0.1723	0.2874	0.1813	0.3893
SR	0.5733	0.7268	0.6573	0.7589	0.2934	0.3162	0.2730	0.5731	0.2960	0.5387	0.4609	0.6252	0.1272	0.4193	0.2904	0.5415
MatrixProfile	0.0574	0.1419	0.0710	0.3665	0.0000	0.0222	0.0073	0.3421	0.0000	0.0145	0.0031	0.2950	0.0000	0.0042	0.0012	0.2430
Sub_PCA	0.0290	0.1073	0.0722	0.3433	0.4357	0.4440	0.3869	0.3944	0.0871	0.1836	0.1548	0.3527	0.0000	0.0365	0.0080	0.2530
CNN	0.4081	0.6195	0.5729	0.6609	0.2140	0.3133	0.2539	0.5120	0.3081	0.5279	0.4323	0.6507	0.1746	0.4151	0.3225	0.5371
LSTMAD	0.4256	0.6420	0.5949	0.6937	0.1609	0.2870	0.2271	0.5349	0.4218	0.6089	0.5199	0.6638	0.1842	0.4149	0.3189	0.5169
M2N2	0.2441	0.5194	0.4614	0.5780	0.1836	0.3161	0.2546	0.4979	0.2163	0.4834	0.4015	0.5965	0.1605	0.4251	0.3395	0.5283
OmniAnomaly	0.1456	0.2493	0.2092	0.4034	0.3999	0.4709	<u>0.4165</u>	0.3889	0.2259	0.3339	0.2984	0.4475	0.0694	0.4960	0.3840	0.4932
TranAD	0.0264	0.0664	0.0410	0.2528	0.2868	0.3137	0.2362	0.3342	0.1307	0.1913	0.1283	0.3556	0.0348	0.0368	0.0195	0.3345
Chronos	0.6983	0.7893	0.7819	0.8634	0.1851	0.2839	0.2199	0.4926	0.3047	0.5146	0.4100	0.5807	0.1485	0.4879	0.3750	0.6086
TimesFM	0.6810	0.8489	0.8259	<u>0.8956</u>	0.1972	0.3121	0.2445	0.5379	0.3548	0.5456	0.4288	0.6027	0.1467	0.4347	0.3502	0.5425
OFA	0.0554	0.1973	0.1357	0.3695	0.1317	0.2603	0.1110	0.3994	0.2253	0.4428	0.2891	0.5201	0.1656	0.2236	0.1021	0.3539
LLM-AD	0.3515	0.3708	0.2195	0.3620	0.3810	0.4171	0.3269	0.5378	0.2929	0.2942	0.1930	0.3723	0.2771	0.3537	0.2558	0.5329
LLM-TSAD	0.6263	0.6263	0.5529	0.6687	0.4373	0.4373	0.2579	0.5266	0.4224	0.4224	0.2892	0.4702	0.3162	0.3162	0.1530	0.4647
ARGOS	0.4835	0.4835	0.5093	0.5902	0.4771	0.4771	0.2877	0.5479	0.3639	0.3639	0.3582	0.4081	0.3009	0.3009	0.2669	0.2356
Ours (In-domain)	0.9015	0.9086	0.8802	0.9028	0.6791	0.6824	0.5192	0.8042	<u>0.6974</u>	<u>0.7019</u>	<u>0.5871</u>	<u>0.7211</u>	<u>0.8005</u>	<u>0.8143</u>	<u>0.6721</u>	<u>0.8398</u>
Ours (Cross-domain)	<u>0.8906</u>	<u>0.8956</u>	<u>0.8562</u>	<u>0.8924</u>	<u>0.6058</u>	<u>0.6179</u>	<u>0.4025</u>	<u>0.7995</u>	0.8049	0.8155	0.7295	0.8422	0.8553	0.8649	0.7671	0.9090

TABLE IV
ABLATION STUDY OF SUPERVISED FINE-TUNING (SFT) AND REINFORCEMENT LEARNING (RL) ACROSS DIFFERENT DATASETS.

Model	YAHOO		KPI		WSD		IOPS	
	F1	Range-F1	F1	Range-F1	F1	Range-F1	F1	Range-F1
w/o Both	0.2201	0.5283	0.0652	0.4238	0.4036	0.5575	0.4004	0.6733
w/o RL	<u>0.8588</u>	<u>0.8679</u>	<u>0.5961</u>	<u>0.7963</u>	<u>0.7911</u>	<u>0.8208</u>	<u>0.8009</u>	<u>0.8202</u>
w/o SFT	0.6070	0.6609	0.3343	0.6732	0.6909	0.6895	0.7279	0.7967
Ours	0.8906	0.8924	0.6058	0.7995	0.8049	0.8422	0.8553	0.9090

TABLE V
ABLATION STUDY OF THE DIFFERENT DETECTION TOOL VARIANTS ACROSS DIVERSE DATASETS.

Model	YAHOO		KPI		WSD		IOPS	
	F1	Range-F1	F1	Range-F1	F1	Range-F1	F1	Range-F1
w/o Feature Extraction	0.8746	0.8710	0.5693	0.7379	0.6983	0.7116	0.8421	0.8201
w/o Value-based Detection	0.8631	0.8631	0.5284	0.6968	0.6470	0.6932	0.7963	0.7863
w/o Change-based Detection	0.7708	0.7708	0.5871	0.7641	<u>0.7347</u>	0.7237	0.8081	0.8095
w/o Knowledge Memory	0.8454	0.8454	0.6102	<u>0.7842</u>	0.7203	<u>0.7532</u>	<u>0.8451</u>	<u>0.8274</u>
Ours	0.8906	0.8924	<u>0.6058</u>	0.7995	0.8049	0.8155	0.8553	0.9090

adapt large language models to TSAD through prompting, alignment, or in-context reasoning.

3) *Implementation Details*: We use Qwen3-8B [44] as the trainable backbone of the detection-specific policy and Gemini-3.1-flash as the general-purpose LLM for coarse localization, tool orchestration, and iterative refinement. The detection policy is optimized within the verl framework with a learning rate of 5×10^{-6} and a batch size of 16 on four NVIDIA A800 GPUs (80GB). We follow official implementations and hyperparameters for all baselines. AnomaMind segments each series into non-overlapping windows, with window sizes of 100 for Yahoo, 500 for WSD, and 1000 for KPI and IOPS. We evaluate AnomaMind under in-domain and cross-domain settings. The in-domain setting trains a separate model for each dataset, while the cross-domain setting trains a unified model on all training splits and evaluates it separately on each test split. Unless otherwise specified, ablation and exploration analyses use the cross-domain setting. We use F1 as the primary metric and additionally report Best-F1, AUC-PR, and Range-F1. For Best-F1 and AUC-PR, we sweep the confidence threshold of predicted intervals and compute the corresponding optimal F1 and precision-recall curve.

B. Main Results

As shown in Table III, AnomaMind achieves strong performance across diverse anomaly detection scenarios. The in-domain variant performs better on YAHOO and KPI, show-

ing its ability to capture dataset-specific anomaly characteristics. The cross-domain variant achieves stronger results on WSD and IOPS, suggesting that unified training across heterogeneous datasets improves generalization across diverse anomaly patterns. Compared with conventional anomaly detectors, AnomaMind maintains more stable detection quality across datasets. This indicates that its coarse-to-fine diagnostic workflow can better handle diverse anomaly shapes and durations. Notably, AnomaMind achieves strong Best-F1 and Range-F1 scores under both in-domain and cross-domain settings, showing that its interval-level decisions remain effective across point-wise and range-level evaluation. Overall, these results demonstrate the effectiveness of AnomaMind’s agentic diagnostic workflow with contextual guidance, numerical diagnostics, tool interaction, and iterative verification.

C. Ablation Studies

1) *Training Strategy Ablation*: As shown in Table IV, the full AnomaMind achieves the best overall performance, demonstrating the effectiveness of combining supervised fine-tuning (SFT) with reinforcement learning (RL). Removing both stages leads to a clear performance drop, indicating that direct prompting without task-specific optimization is insufficient for reliable anomaly detection. Compared with the full model, removing RL reduces detection performance, suggesting that RL helps refine the decision policy and improve stability. Removing SFT causes a larger degradation,

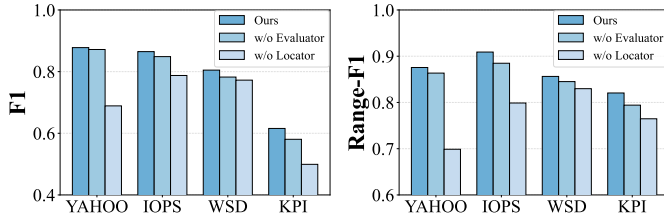


Fig. 3. Ablation study on the effectiveness of key agent roles in AnomaMind across F1 and Range-F1 metrics.

TABLE VI

IMPACT OF BACKBONE MODEL SIZE ON ANOMALY DETECTION PERFORMANCE UNDER SPECIALIZED REINFORCEMENT LEARNING.

Model	YAHOO		KPI		WSD		IOPS	
	F1	Range-F1	F1	Range-F1	F1	Range-F1	F1	Range-F1
Qwen3-0.6B	0.8271	0.8325	0.5332	0.7374	0.7054	0.7629	0.8061	0.8114
Qwen3-1.7B	0.8524	0.8536	0.5232	0.7511	0.6987	0.7575	0.8027	0.8069
Qwen3-4B	0.8695	0.8667	0.5747	0.8015	0.7283	0.7666	0.8371	0.8947
Qwen3-8B	0.8906	0.8924	0.6058	0.7995	0.8049	0.8422	0.8553	0.9090

showing that SFT provides essential task-aligned initialization for understanding anomaly evidence and detection objectives. Overall, SFT and RL play complementary roles: SFT builds the basic anomaly reasoning capability, while RL further enhances decision quality through feedback-driven optimization.

2) *Detection Tool Ablation*: As shown in Table V, the full AnomaMind achieves the best overall performance, demonstrating the effectiveness of the proposed detection tools. Removing individual tool components generally leads to performance degradation, indicating that different tools provide complementary evidence for anomaly detection. In particular, change-based detection captures abnormal temporal transitions, while value-based detection identifies magnitude-level deviations. Feature extraction provides explicit statistical evidence for characterizing anomaly patterns, and knowledge memory offers contextual grounding with reusable prior knowledge. These results confirm that coordinated tool interaction is essential for coarse-to-fine anomaly detection.

3) *Workflow Component Ablation*: As shown in Figure 3, removing either the evaluator or the locator degrades performance, demonstrating the necessity of key agent roles in AnomaMind. Removing the evaluator causes a moderate but consistent drop, indicating that decision verification and iterative refinement help reduce unstable predictions. The degradation is more pronounced when the locator is removed, suggesting that accurate coarse localization is crucial for identifying informative candidate regions and providing reliable evidence for reasoning. Without effective localization, later detection steps may receive noisy evidence, leading to accumulated errors and weaker range-level localization. Overall, these results confirm that the locator and evaluator provide complementary support: the locator improves evidence acquisition, while the evaluator enhances decision reliability.

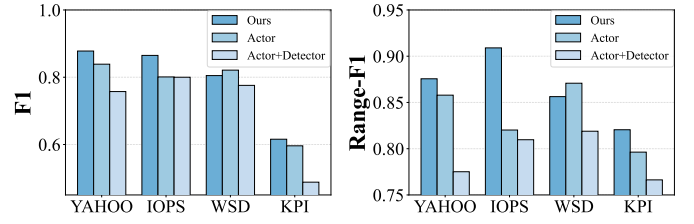


Fig. 4. Performance comparison of different trained agent components across F1 and Range-F1 metrics.

TABLE VII

EXPERIMENTAL RESULTS WITH DIFFERENT GENERAL-PURPOSE MODEL VARIANTS UNDER THE PROPOSED FRAMEWORK.

Model	YAHOO		KPI		WSD		IOPS	
	F1	Range-F1	F1	Range-F1	F1	Range-F1	F1	Range-F1
Gemini-3.1-flash	0.8906	0.8924	0.6058	0.7995	0.8049	0.8422	0.8553	0.9090
Grok-4.1	0.8482	0.8601	0.4244	0.7485	0.8363	0.8562	0.8706	0.9093
GPT-5-mini	0.7735	0.8530	0.4474	0.6911	0.7661	0.8398	0.7398	0.7959

D. Exploration Analysis

1) *Impact of Different Trained Agent Components*: As shown in Figure 4, the full AnomaMind achieves the strongest overall performance, indicating the importance of coordinated training across agent components. Training only the actor yields reasonable results, suggesting that it learns useful evidence construction and reasoning behaviors, but actor-level reasoning alone remains insufficient for stable detection. Jointly training the actor and detector does not consistently improve over the actor-only variant, indicating that strengthening detection without the complete workflow may lead to unstable or insufficiently verified decisions. In contrast, the full AnomaMind benefits from coordinated evidence construction, detection, and refinement. Overall, specialized component training is useful, but robust anomaly detection depends on its integration within the complete reasoning workflow.

2) *Impact of Backbone Size*: Model scale has a clear impact on AnomaMind under specialized reinforcement learning. As shown in Table VI, Qwen3-8B achieves the best overall performance, demonstrating that a larger backbone provides stronger reasoning and anomaly-discrimination capacity. Compared with smaller variants, it better supports complex anomaly reasoning and interval-level localization. The results also show that performance does not increase monotonically from 0.6B to 1.7B, indicating that moderate scaling alone does not guarantee improvement. However, increasing the backbone size to 4B and 8B leads to more consistent gains, suggesting that sufficient model capacity is important for learning task-specific decision policies through reinforcement learning.

3) *Impact of General-Purpose Models*: As shown in Table VII, the proposed framework maintains competitive performance across different general-purpose model variants, indicating that it is not tightly coupled to a specific backbone. Gemini-3.1-flash achieves stronger results on YAHOO and KPI, while Grok-4.1 performs better on WSD and IOPS, suggesting that different models provide complementary reasoning strengths across datasets. In comparison, GPT-5-mini

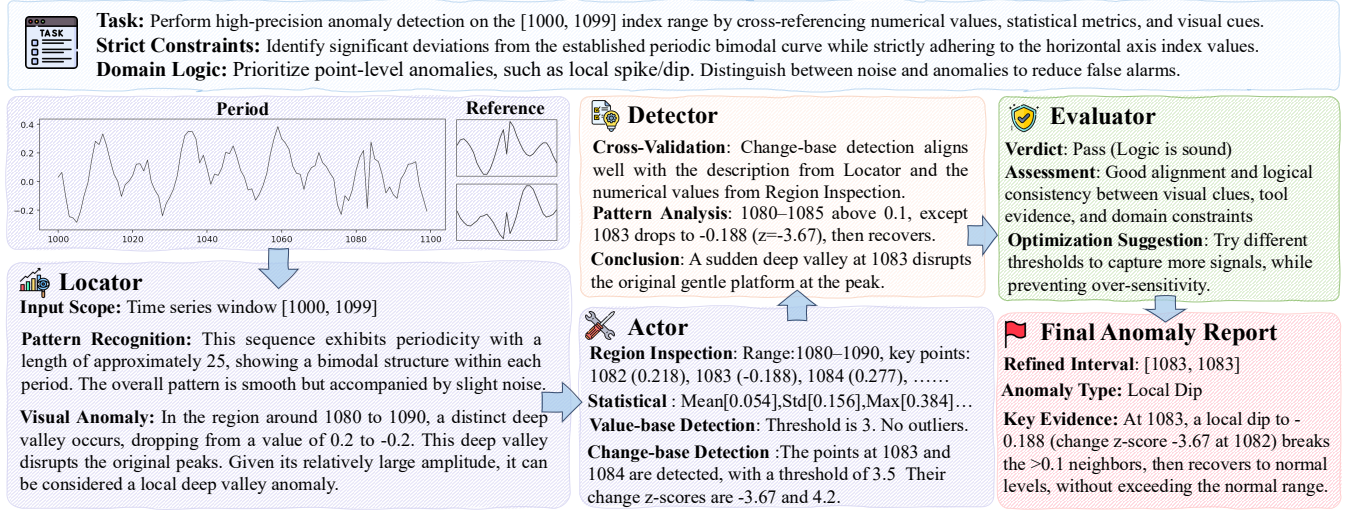


Fig. 5. An illustrative example of a structured coarse-to-fine workflow for agentic time series anomaly detection.

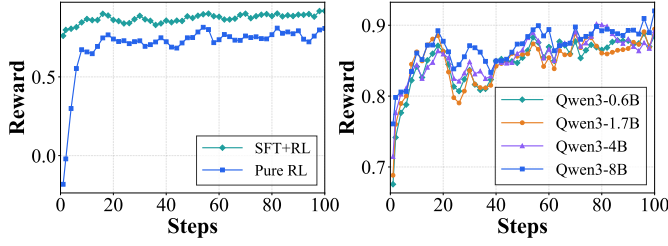


Fig. 6. Training convergence analysis under different optimization strategies and backbone model scales.

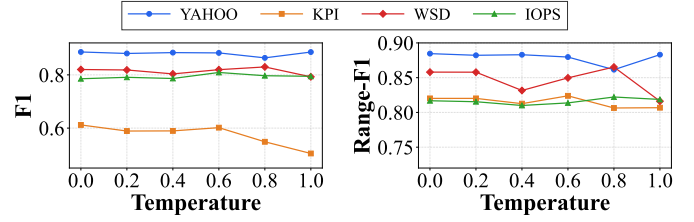


Fig. 7. Sensitivity analysis of sampling temperature on anomaly detection performance across F1 and Range-F1 metrics.

remains effective but performs relatively lower, indicating that lighter models may have limited capacity for complex anomaly discrimination. Overall, these results demonstrate the general applicability of the proposed framework, where stronger backbones tend to provide more stable anomaly reasoning and interval-level localization.

4) *Impact of Training Convergence:* As shown in Figure 6, AnomaMind shows stable convergence across training strategies and backbone scales. Compared with pure RL, SFT+RL achieves higher early rewards and a steadier upward trend, indicating that supervised fine-tuning provides a better task-aligned initialization. Pure RL converges more slowly due to weaker initial task alignment. Different backbone models also converge smoothly. Larger models generally obtain slightly higher and more stable rewards, while smaller models still improve consistently. These results show that the training process is effective across model scales and that SFT initialization improves optimization stability and efficiency.

5) *Sampling Temperature Sensitivity:* As shown in Figure 7, we evaluate the sensitivity of AnomaMind to sampling temperature. AnomaMind maintains stable performance across a wide temperature range, indicating reliable interval-level localization without careful hyperparameter tuning. For F1-score, performance shows only mild fluctuations under most

temperatures, while overly high temperatures may slightly weaken point-wise detection due to increased randomness. Low-to-moderate temperatures generally provide more stable results by balancing deterministic reasoning and limited exploration. Overall, these results demonstrate that AnomaMind is robust to sampling temperature variations.

E. Case Studies

Figure 5 illustrates a representative coarse-to-fine detection case. For a periodic bimodal time series, the locator identifies a suspicious local valley around indices 1080–1090. The actor then performs tool-assisted inspection: value-based detection finds no clear amplitude outliers, while change-based detection reveals sharp transitions around indices 1083 and 1084. The detector confirms an anomaly at index 1083, and the evaluator verifies evidence consistency. Finally, the framework outputs the refined interval [1083, 1083], showing precise detection through multi-stage reasoning and tool interaction.

VI. CONCLUSION

In this work, we propose AnomaMind, an agentic TSAD framework that reformulates anomaly detection as an evidence-driven sequential decision-making process. AnomaMind follows a coarse-to-fine workflow that localizes suspicious intervals, constructs diagnostic evidence through tool in-

teraction, and refines decisions through self-reflection. Anoma-Mind is supported by a toolkit box that integrates knowledge memory and numerical diagnostics. It further adopts a hybrid inference mechanism, where general-purpose models handle flexible reasoning and tool invocation, while a detection-specific policy is optimized with rule-based rewards. Extensive experiments under both in-domain and cross-domain settings demonstrate consistent performance gains, validating the effectiveness of tool-augmented reasoning for TSAD.

REFERENCES

- [1] Q. Liu and J. Paparrizos, “The elephant in the room: Towards a reliable time-series anomaly detection benchmark,” in *NeurIPS 2024*, 2024.
- [2] X. Qiu, Z. Li, W. Qiu, S. Hu, L. Zhou, X. Wu, Z. Li, C. Guo, A. Zhou, Z. Sheng, J. Hu, C. S. Jensen, and B. Yang, “Tab: Unified benchmarking of time series anomaly detection methods,” *Proceedings of the VLDB Endowment*, vol. 18, pp. 2775–2789, 2025.
- [3] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM computing surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.
- [4] M. Munir, S. A. Siddiqui, A. Dengel, and S. Ahmed, “Deepant: A deep learning approach for unsupervised anomaly detection in time series,” *Ieee Access*, vol. 7, pp. 1991–2005, 2018.
- [5] X. Tao, M. Cheng, C. Jiang, T. Gao, H. Zhang, and Y. Liu, “Cast-r1: Learning tool-augmented sequential decision policies for time series forecasting,” *arXiv preprint arXiv:2602.13802*, 2026.
- [6] T. Zhou, P. Niu, L. Sun, R. Jin *et al.*, “One fits all: Power general time series analysis by pretrained llm,” *Advances in neural information processing systems*, vol. 36, pp. 43 322–43 355, 2023.
- [7] Z. Zhou and R. Yu, “Can llms understand time series anomalies?” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 1858–1896.
- [8] M. Cheng, X. Tao, Q. Liu, Z. Guo, and E. Chen, “Position: Beyond model-centric prediction—agentic time series forecasting,” *arXiv preprint arXiv:2602.01776*, 2026.
- [9] Z. Gu, B. Zhu, G. Zhu, Y. Chen, M. Tang, and J. Wang, “Anomalygpt: Detecting industrial anomalies using large vision-language models,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 3, 2024, pp. 1932–1940.
- [10] Y. Jiang, Z. Pan, X. Zhang, S. Garg, A. Schneider, Y. Nevmyvaka, and D. Song, “Empowering time series analysis with large language models: a survey,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 8095–8103.
- [11] A. Yang, Y. Chen, S. Lee, and V. Montes, “Refining time series anomaly detectors using large language models,” *arXiv preprint arXiv:2503.21833*, 2025.
- [12] X. Zhang, T. Gao, M. Cheng, B. Pan, Z. Guo, Y. Liu, and X. Tao, “Alphacast: A human wisdom-llm intelligence co-reasoning framework for interactive time series forecasting,” *arXiv preprint arXiv:2511.08947*, 2025.
- [13] X. Wu, J. Lu, Z. Li, X. Qiu, J. Hu, C. Guo, C. S. Jensen, and B. Yang, “Timeart: Towards agentic time series reasoning via tool-augmentation,” *arXiv preprint arXiv:2601.13653*, 2026.
- [14] J. Xu, H. Wu, J. Wang, and M. Long, “Anomaly transformer: Time series anomaly detection with association discrepancy,” in *International Conference on Learning Representations*, 2022.
- [15] J. Park, K. Jung, D. Lee, H. Lee, D. Gwak, C. Park, J. Choo, and J. Cho, “Delving into large language models for effective time-series anomaly detection,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [16] S. W. Roberts, “Control chart tests based on geometric moving averages,” *Technometrics*, vol. 42, no. 1, pp. 97–101, 2000.
- [17] P. J. Rousseeuw and A. M. Leroy, *Robust regression and outlier detection*. John Wiley & sons, 2003.
- [18] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “Lof: identifying density-based local outliers,” in *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, 2000, pp. 93–104.
- [19] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, “Robust anomaly detection for multivariate time series through stochastic recurrent neural network,” in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2828–2837.
- [20] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, “Usad: Unsupervised anomaly detection on multivariate time series,” in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 3395–3404.
- [21] K. Faber, M. Pietron, D. Zurek, and R. Corizzo, “xlstmad: A powerful xlstm-based method for anomaly detection,” in *2025 IEEE International Conference on Data Mining (ICDM)*, 2025, pp. 247–256.
- [22] A. Bhat, A. Kumar, A. Chekodu, B. Chandana, and S. Shylaja, “Enhanced anomaly detection in time-series data: A comparative study of univariate approach with transformer and llm methods,” in *International Conference on Information and Communication Technology for Intelligent Systems*. Springer, 2025, pp. 261–271.
- [23] Y. Liu, Y. Luo, X. Li, X. Dong, B. Gu, and Z. Jin, “Evaluating large language models for time series anomaly detection in aerospace software,” in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 3522–3533.
- [24] J. Tang, S. Chen, C. Gong, J. Zhang, and D. Tao, “Empowering large language models for time series forecasting with patterns and semantics,” in *2025 IEEE International Conference on Data Mining (ICDM)*, 2025.
- [25] A. Bechar, A. Oulefki, A. Amira, F. Kurogollu, and Y. Himeur, “Extracting actionable insights from building energy data using vision llms on wavelet and 3d recurrence representations,” in *2025 IEEE International Conference on Data Mining (ICDM)*, 2025.
- [26] C. Liu, S. He, Q. Zhou, S. Li, and W. Meng, “Large language model guided knowledge distillation for time series anomaly detection,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 2162–2170.
- [27] Y. Gu, Y. Xiong, J. Mace, Y. Jiang, Y. Hu, B. Kasikci, and P. Cheng, “Argos: Agentic time-series anomaly detection with autonomous rule generation via large language models,” *arXiv preprint arXiv:2501.14170*, 2025.
- [28] A. Chandrayan, Z. Amir, M. Reimherr, A. Mjirda, and A. Pradhan, “Lead-framework for efficient time-series anomaly detection on large scale data using llms,” in *1st ICML Workshop on Foundation Models for Structured Data*.
- [29] X. Tao, M. Cheng, Z. Guo, S. Yu, Y. Liu, Q. Liu, and S. Wang, “Memcast: Memory-driven time series forecasting with experience-conditioned reasoning,” *arXiv preprint arXiv:2602.03164*, 2026.
- [30] C. Jiang, M. Cheng, X. Tao, Q. Mao, J. Ouyang, and Q. Liu, “Tablemind: An autonomous programmatic agent for tool-augmented table reasoning,” in *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*, 2026, pp. 260–270.
- [31] N. Laptev, S. Amizadeh, and I. Flint, “Generic and scalable framework for automated time-series anomaly detection,” in *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, 2015, pp. 1939–1947.
- [32] S. Zhang, Z. Zhong, D. Li, Q. Fan, Y. Sun, M. Zhu, Y. Zhang, D. Pei, J. Sun, Y. Liu *et al.*, “Efficient kpi anomaly detection through transfer learning for large-scale web services,” *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 8, pp. 2440–2455, 2022.
- [33] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, D. Pei, Y. Feng *et al.*, “Unsupervised anomaly detection via variational auto-encoder for seasonal kpis in web applications,” in *Proceedings of the 2018 world wide web conference*, 2018, pp. 187–196.
- [34] M. Thill, W. Konen, and T. Bäck, “Time series anomaly detection with discrete wavelet transforms and maximum likelihood estimation,” in *Proceedings of the International Conference on Time Series and Forecasting*, vol. 2, 2017, pp. 11–23.
- [35] C.-C. M. Yeh, Y. Zhu, L. Ulanova, N. Begum, Y. Ding, H. A. Dau, D. F. Silva, A. Mueen, and E. Keogh, “Matrix profile i: All pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets,” in *2016 IEEE 16th International Conference on Data Mining*. IEEE, 2016, pp. 1317–1322.
- [36] H. Ren, B. Xu, Y. Wang, Q. Yi *et al.*, “Time-series anomaly detection service at microsoft,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 3009–3017.
- [37] M.-L. Shyu, S.-C. Chen, K. Sarinapakorn, and L. Chang, “A novel anomaly detection scheme based on principal component classifier,” in *Proceedings of the IEEE Foundations and New Directions of Data Mining Workshop*, 2003, pp. 172–179.
- [38] D. Kim, S. Park, and J. Choo, “When model meets new normals: Test-time adaptation for unsupervised time-series anomaly detection,” in

Proceedings of the AAAI conference on artificial intelligence, vol. 38, no. 12, 2024, pp. 13 113–13 121.

- [39] P. Malhotra, L. Vig, G. Shroff, P. Agarwal *et al.*, “Long short term memory networks for anomaly detection in time series,” in *Proceedings*, vol. 89, no. 9, 2015, p. 94.
- [40] S. Tuli, G. Casale, and N. R. Jennings, “Tranad: deep transformer networks for anomaly detection in multivariate time series data,” *Proceedings of the VLDB Endowment*, vol. 15, no. 6, pp. 1201–1214, 2022.
- [41] A. Das, W. Kong, R. Sen, and Y. Zhou, “A decoder-only foundation model for time-series forecasting,” in *Forty-first International Conference on Machine Learning*, 2024.
- [42] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. P. Arango, S. Kapoor *et al.*, “Chronos: Learning the language of time series,” *arXiv preprint arXiv:2403.07815*, 2024.
- [43] J. Liu, C. Zhang, J. Qian, M. Ma, S. Qin, C. Bansal, Q. Lin, S. Rajmohan, and D. Zhang, “Large language models can deliver accurate and interpretable time series anomaly detection,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 4623–4634.
- [44] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.

APPENDIX A DATASET DETAILS

We evaluate our method on four widely used time series anomaly detection benchmarks covering production traffic, service KPIs, and system-level monitoring measurements, with both point-level and range-level anomaly annotations. Details are provided below:

- **YAHOO**: The Yahoo S5 benchmark contains one real-world subset from Yahoo production traffic and three synthetic subsets. It covers diverse anomaly types, including point, contextual, and collective anomalies, and is widely used for univariate time series anomaly detection.
- **KPI**: The KPI benchmark contains operational metrics from large-scale web services, with segment-level anomaly labels. Its anomalies often appear as short abnormal intervals within noisy and periodic service behavior.
- **IOPS**: The IOPS benchmark contains real-world system-level monitoring metrics from cloud and web services. It includes labeled segment anomalies such as spikes, drops, and drifts, making it suitable for evaluating anomaly detection in complex IT environments.
- **WSD**: The WSD benchmark consists of KPIs from large-scale internet services, including traffic, response time, and error rate metrics. It features production noise, strong periodic patterns, and low anomaly density ($\sim 0.6\%$), with labeled spikes, contextual shifts, and long-term drifts.

APPENDIX B BASELINE DETAILS

To comprehensively evaluate our method, we benchmark against competitive approaches covering statistical and classical machine learning methods, deep learning models, foundation models, and LLM-based anomaly detectors.

A. Statistical and Classical Approaches

DWT-MLEAD applies wavelet decomposition to model time series at multiple resolutions and detects anomalies from coefficient- or reconstruction-level deviations. **FFT-AD**

decomposes time series into frequency components and identifies anomalies through large high-frequency residuals after low-frequency reconstruction. **MatrixProfile** computes subsequence similarity profiles and flags subsequences with unusually large nearest-neighbor distances. **SR** (Spectral Residual) uses Fourier-based log-amplitude deviations to construct a saliency map for point-level anomaly detection. **Sub-PCA** applies PCA to subsequence representations and detects anomalies from reconstruction deviations from normal subspaces.

B. Deep Learning Baselines

CNN employs 1D convolutional layers to extract local temporal features and forecasts future values via fully connected layers, detecting anomalies based on prediction error thresholds. **M2N2** is a test-time adaptation method that handles distribution shifts. It detrends sequences via exponential moving averages and selectively updates parameters on low-error instances through self-supervision. **LSTMAD** utilizes an LSTM-based encoder-decoder architecture to learn latent representations of normal patterns. It identifies anomalies by thresholding reconstruction errors, as deviations yield significantly higher losses. **TranAD** leverages a Transformer encoder-decoder with multi-head self-attention to capture global temporal dependencies, using reconstruction error for anomaly detection. **OmniAnomaly** models stochastic latent representations of multivariate temporal dynamics and detects anomalies through reconstruction likelihoods.

C. Foundation Models

TimesFM is a decoder-only foundation model that patches sequences into fixed-length tokens. It autoregressively generates future values, detecting anomalies via point-wise prediction errors. **Chronos** is a language-model-based forecaster that tokenizes time series. Operating in a zero-shot setup, it produces probabilistic forecasts via a transformer encoder-decoder, flagging anomalies based on prediction deviations.

D. LLM-based Approaches

LLM-TSAD decomposes time series into trend, seasonal, and residual components, converting them into text descriptions. Using few-shot prompts, the LLM reasons over the decomposed signals to identify anomalies. **LLM-AD** employs LLM reasoning for anomaly detection through textualized temporal observations and task-specific prompts. **OFA** (One Fits All) utilizes a frozen pretrained transformer backbone with lightweight task-specific adapters. It processes sequences through the frozen encoder and uses adapter heads to reconstruct inputs for anomaly identification. **ARGOS** adapts large language models to TSAD through anomaly-oriented prompting, alignment, or in-context reasoning.

APPENDIX C

EXTENDED ANALYSIS AND EXPERIMENTAL RESULTS

In this section, we provide additional analyses to further validate the effectiveness of AnomaMind. We first present qualitative visualization comparisons to show its precision in

anomaly localization against baseline methods. We then provide the prompts used in AnomaMind to clarify the reasoning and detection process.

A. Visualization Case

We visualize the detection results of our method alongside baseline approaches in Figure 8. Observations indicate that while baselines such as FFT, TimesFM, M2N2, LLM-TSAD, Sub PCA, and OmniAnomaly successfully capture all anomalous regions, they suffer from a notably high false positive rate. Conversely, other baselines—including TranAD, ARGOS, CNN, OFA, and LSTMAD—produce fewer false alarms but fail to detect the second anomalous high plateau, often flagging only the first spike. MatrixProfile and DWT-MLEAD partially localize the second anomaly yet still incur false positives in neighboring normal intervals. In contrast, our approach, empowered by tool-augmented reasoning, accurately identifies all anomalous intervals with zero false positives, demonstrating superior precision and accuracy.

APPENDIX D

ADDITIONAL IMPLEMENTATION DETAILS

A. Data Preprocessing

Time series data often exhibit significant variations in numerical magnitude and timestamp formats. To ensure accurate anomaly pattern recognition and enhance model stability, we implement the following preprocessing steps:

1) *Normalization*: We normalize numerical values to the range $[0, 1]$ via min-max scaling:

$$x' = \frac{x - a}{b - a}$$

where a and b represent the minimum and maximum values of the time series, respectively.

2) *Indexing*: To mitigate the inconsistencies caused by varying timestamp formats and sampling frequencies across datasets, we abstract absolute time into a unified zero-based integer index. Formally, we map the raw time series into an ordered sequence of index-value pairs $\mathcal{X} = \{(t, v_t)\}_{t=0}^{T-1}$, where t represents the relative temporal step and T denotes the sequence length. This standardized representation ensures that the agents focus on relative temporal patterns and output consistent interval coordinates (e.g., $[t_{start}, t_{end}]$) regardless of the original time format.

3) *Detrending*: To mitigate the influence of global trends that are irrelevant to local anomalies, we remove the linear component:

$$x' = x - \hat{x}_{\text{trend}}$$

Here, $\hat{x}_{\text{trend}}$ denotes the linear trend estimated via least squares regression.

B. Evaluation Metrics

1) *F1-Score*: The primary goal of time series anomaly detection is to identify anomalous points while minimizing false alarms. We adopt the standard definitions for Recall, Precision, and F1-score:

- **Recall**: Measures the proportion of actual anomalies that are correctly detected. It assesses the model’s ability to capture all relevant instances.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **Precision**: Quantifies the reliability of the reported anomalies. It indicates the ratio of true anomalies among all positive predictions, reflecting the model’s ability to avoid false alarms.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **F1-score**: Provides a balanced metric of overall detection performance. It is calculated as the harmonic mean of Precision and Recall, making it suitable for scenarios with imbalanced data.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

For traditional models that output continuous anomaly scores, we apply the widely adopted threshold of $\mu + 3\sigma$; points exceeding this threshold are classified as anomalous. For LLM-based models, points are considered anomalous if they fall within the intervals explicitly generated by the model.

2) Best-F1:

- **Traditional methods**: Best-F1 represents the maximum F1-score obtained by sweeping over possible anomaly score thresholds.
- **Our method**: We classify intervals as anomalous only if their associated confidence score meets or exceeds a specific threshold. Consequently, Best-F1 is determined by finding the maximum F1-score across all possible confidence thresholds.
- **Other LLM-based methods**: We adhere to the Best-F1 calculation procedures specified in their respective implementations.

3) *AUC-PR and Range-F1*: In addition to F1 and Best-F1, we report AUC-PR and Range-F1 in the main experiments. AUC-PR is computed from the precision-recall curve obtained by sweeping the confidence threshold of predicted intervals, which reflects detection quality under class imbalance. Range-F1 evaluates interval-level localization by accounting for overlap between predicted and ground-truth anomalous ranges, making it suitable for KPI, IOPS, and WSD, where anomalies are mainly annotated as segments.

C. Prompt Templates

To facilitate reproducibility and provide insight into the underlying reasoning mechanisms of our framework, we present the detailed prompt templates designed for each agentic component. These structured instructions serve as the cognitive blueprint for the system, strictly defining the functional roles, operational constraints, and expected interaction protocols. By standardizing the input-output formats and reasoning pathways, these prompts ensure that the workflow progresses coherently from coarse-grained visual perception to fine-grained evidence analysis.

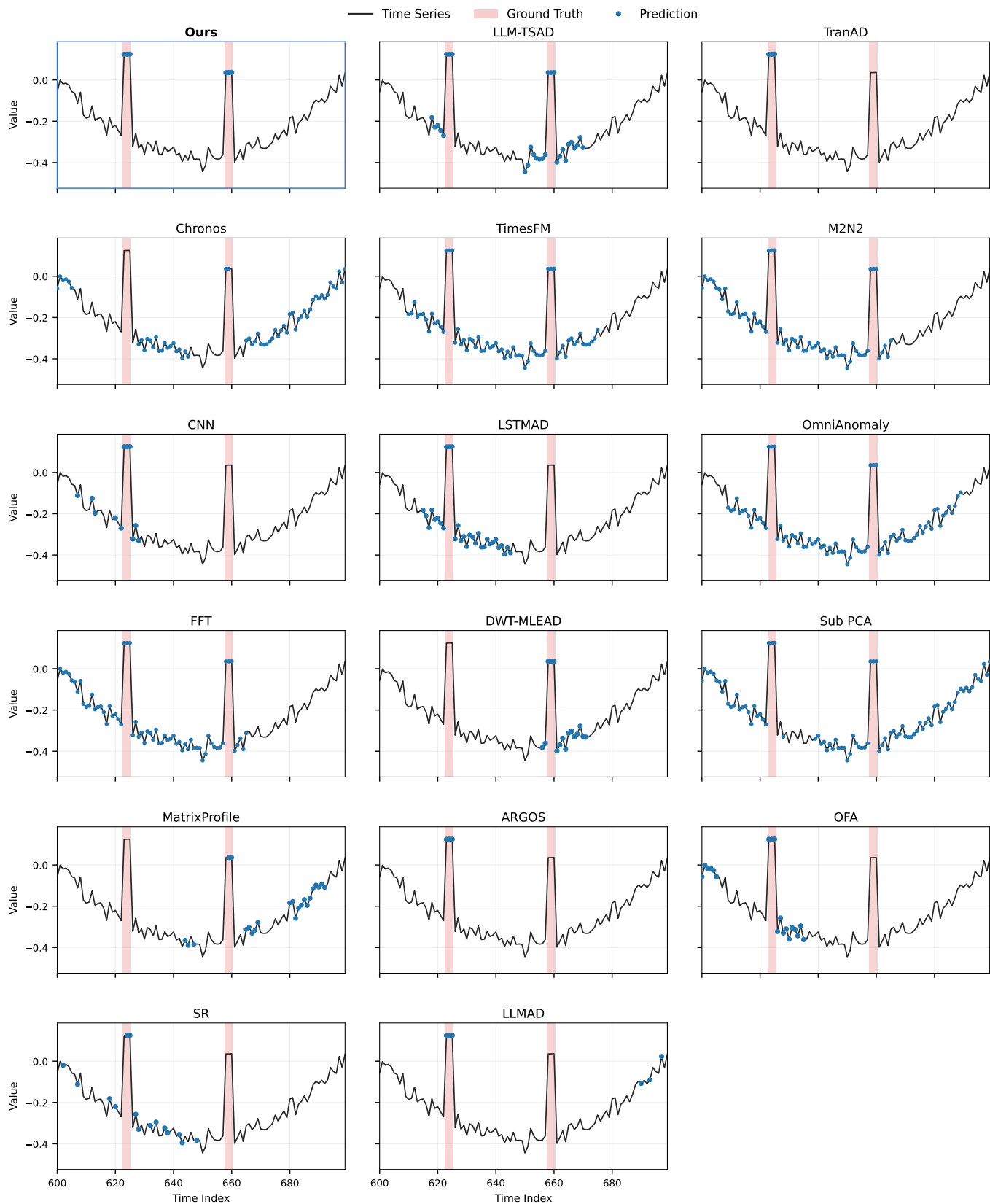


Fig. 8. Visualization of detection results demonstrating the superior precision of AnomaMind compared to leading baselines.

Locator Prompt

Role:

Input Context: You are an anomaly detection expert. You will be given a time series line chart and relevant typical anomaly pattern diagrams. You need to analyze whether the line chart contains any anomalous intervals and compare them with the patterns:

- **Picture:** The time series picture, with anomaly pattern visual references. {Picture}.
- **Knowledge:** Domain-specific constraints and patterns provided in {Domain Knowledge}.

Guidelines: Adhere to the following rules:

- **Completeness:** Do not miss any suspicious intervals. Include as many suspicious intervals as possible.
- **Constraint:** Anomaly intervals should be identified based on domain-specific typical anomaly patterns and domain knowledge.

Output Protocol: Structure your response in two strict parts:

- <Visual Description>: Your visual summary of the line chart, including information on shape, value fluctuation range, and periodicity.
- <Intervals>: The Candidate Intervals detected from picture.
- <Rationale>: Explanation of why each candidate interval is considered an anomaly.

Actor Prompt

Role: You are a time series anomaly detection expert responsible for tool execution. Your primary function is to call right tools based on coarse candidate intervals and domain knowledge.

Input Context:

- **Knowledge:** The description and rules of dataset. {Domain Knowledge}.
- **Tool Description:** The explanation, parameters, expected output format, and usage constraints for each available tool. Tool Description. You must strictly follow the input-output specifications defined here. Do not attempt to use a tool in a way that violates its description.
- **Candidate Intervals:** The specious intervals from locator {Candidate Intervals}.

Execution Protocol: Adhere to the following strict rules:

- **Evidence-Driven Stopping Criterion:** Collect as much diverse tool evidence as possible; stop calling only when the evidence is sufficient to determine all anomaly locations. Do not stop early unless every candidate interval has been sufficiently supported.
- **Batch Execution:** Efficiency is paramount. Try to invoke more independent tools in few turns.
- **Compliance:** Do not deviate from the domain rules; focus solely on execution logic.

Detector Prompt

Role: You are the core anomaly detector. Your task is to synthesize tool execution results, the specious intervals, domain knowledge to render a final, reasoned verdict.

Input Context:

- **Execution History:** The results from previous tool calls (`{Tool_Interaction}`).
- **Knowledge:** Domain-specific rules for fine-grained reasoning (`{Domain Knowledge}`).

Response Structure: Your output must strictly follow this format:

- `<think>`: A concise analysis of how the tool results support or refute the anomaly hypothesis.
- **JSON Array:** A list of detected anomalies (or an empty array `[]` if none). Each object must include:
 - `"interval"`: `[start_index, end_index]` (Use actual data indices).
 - `"type"`: The specific name of the anomaly type.
 - `"explanation"`: Concrete reasoning for the decision.
 - `"confidence"`: An integer score from 1 (Low) to 3 (High).

Critical Constraints:

- **Evidence-Based:** Decisions must be grounded in the provided tool results and data values.
- **Index Fidelity:** Strictly strictly adhere to the original data index values.

Evaluator Prompt

Role: You are the final reviewer responsible for quality assurance. Your task is to evaluate the detection results for logical consistency and critical errors.

Input Context:

- **Tool Interaction:** The tool calls and tool results in Actor (`{Tool Interaction}`).
- **Result:** The detection outcome and reasoning provided by the Detector (`{Detector Result}`).

Evaluation Criteria: Adopt a **lenient** evaluation stance. Intervention is required *only* for critical failures.

- **Pass Condition:** If the analysis is generally reasonable, set `needs_refinement` to `false`.
- **Fail Condition:** Trigger refinement (`true`) *only* for major errors (e.g., critical logic flaws, hallucinated data, or complete absence of reasoning).
- **Rating Scale:** Use "poor" strictly for severe issues. Default to "good" or "acceptable" for minor imperfections.

Output Protocol: Return a strict JSON object containing the following keys:

- `"issues"`: A list of strings describing critical errors (if any).
- `"suggestions"`: Actionable advice for correction.
- `"needs_refinement"`: Boolean (`true/false`).
- `"quality_metrics"`: Ratings for `"tool_usage"` and `"reasoning"` (Value: "good"/"acceptable"/"poor").